Path Tracer Final Report

CSSE 490

Jonathan Spychalski, Harvey Yang, Aria Seiler, Aidan Frantz

Overview

We implemented bidirectional path tracing. The main advantage of this technique is that it produces more photorealistic and physically accurate results, given sufficient ray density and depth. Instead of using approximate methods such as the Phong Reflection Model, the actual paths of light rays are traced through the entire scene.

For example, with traditional ray tracing, blocking a light ray from reaching a surface would make it completely black, making shadows sharp and unrealistically dark. In the real world, shadows are softer and their shade varies widely across an image.

We find that while our implementation takes longer than the original ray tracer to get to a comparable image quality, it is able to make out certain details that the standard tracer cannot. For example, the added effect of color bleeding is noticeable, as well as the reflection of light onto the ceiling in the Cornell Box scene.

Implementation

Implemented Features

The core feature we implemented was bidirectional path tracing. However, there are also a number of smaller features that we leverage to improve the quality of our results. We implemented Reinhard shading, which gives more balanced contrast and dynamic range.

Tracing Parameters

`rpp`: The number of rays to trace per pixel of the camera's view
`rpl`: The number of rays to trace per light in the scene
`RES`: The horizontal and vertical resolution of the output image, in pixels
`seed`: Random if 0. Otherwise, ensures determinism by seeding the RNG with this number
`MAX_TRACE_DEPTH`: The maximum number of times a ray will bounce (from a camera or light)

Camera Subpath

Several (`rpp`) rays are traced from each pixel on the image plane, and reflections are followed up to the depth limit or until a reflection does not hit anything.

Light Subpath

Several (`rpl`) rays are traced from each light. Each ray starts at the location of the light and is directed outwards in a random direction; reflections are then followed up to the depth limit (or until a reflection does not hit anything) to create each light subpath.

Ray Bounces

When rays intersect geometry, their next path is computed randomly. If the surface has no reflective component, the resulting ray is a randomly selected point in the hemisphere around the intersection point. If there's a reflective component, the resultant ray has that percent chance to be randomly selected within a lobe around the incoming ray's reflection, or otherwise uses the diffuse logic.

While these random reflections would ideally be uniform, because we were unable to create a seeded gaussian random generator within a reasonable timeframe, they're biased towards the corners of a unit square.

Implementation of the FullPath Class

Sequences of rays are represented as `FullPath` objects, which store the following:

- The starting point's position
- The starting point's material
- Information about each of the hits (bounce points) along the path
- The relative intensity of the hit at each point along the path, usually a representation of how likely the ray is to follow that path

Combining Subpaths

To stitch the camera and light subpaths together, we implemented functions that connect the last hit from a camera subpath to the last hit from a light subpath while reversing the light path, so that the new path is from the camera to the light. If the connection ray is unobstructed, the paths are combined and the connection's strength is the likelihood of the paths having been taken.

In order to further ensure that each ray influences the output and model a rudimentary form of subsurface scattering, if the paths are obstructed, their strength is reduced by the distance of obstruction. This effectively cancels the path if the barrier is large, but if the paths are only separated by a small distance, some of the color can bleed through.

Computing the Final Color

To calculate the final color of a full path from camera to light, we use the diffuse component to compute the color at each point along the path. We did not implement separate coloring for the specular component.

Pixel Colors

The color of a pixel is the average of the colors of each full path traced from it, as calculated above. It is then mapped according to an extended Reinhard mapping, using the maximum component of all pixels as the whitepoint.

Tone Mapping

We implemented extended Reinhard tone mapping, which improves the color balance in scenes. Specifically, it helps make images look natural by trying to maintain as much detail in the highlights and shadows as possible while also keeping reasonable contrast in the midtones. The extended variant utilizes a custom whitepoint to ensure that bright points are kept white.

Instructions for Running

The general run parameter syntax is similar to the original tracer, and the basic usage is `./tracer [options] scene [output]`. The scene is the path to the scene file and the output is the output image file path. If the output is not provided, it defaults to `test.png`.

The options are the following:

- `-r x y` or `-res x y`: Sets the resolution of the output image to `x` by `y`.
- `-p n` or `-rpp n`: Sets the number of rays cast per pixel to `n`.
- `-l` or `-linear`: Uses a slower linear ray intersect, which uses a primitive array.

Usage example:

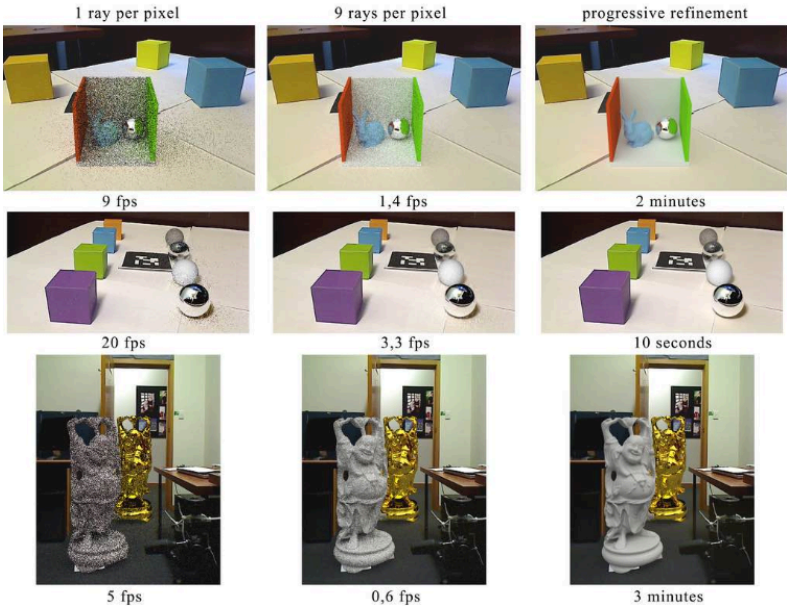
```
./tracer -r 100 100 scene.obj test.png
```

This reads the object file `scene.obj`, generates a 100x100 image at 50 rays per pixel, and write the final image to test.png.

Note that the `rpp` option was occasionally inconsistent, as well as the lack of `rpl`, `seed`, and `depth` commands. It will likely be easier to configure them manually in `Main.cpp` and `RayTracer.h`.

State of the Art

The current state-of-the-art techniques in path tracing are increasingly being integrated into video games, movies, and architectural visualization. One of the latest developments is known as Differential Progressive Path Tracing. This approach builds on traditional path tracing by incorporating real-life environmental lighting into the process, making augmented reality scenes appear much more realistic.



A demonstration of differential progressive path tracing

In practice, it combines the core techniques of path tracing with actual lighting data from the environment, allowing virtual objects to merge seamlessly into real-world images. This integration means that digital elements not only appear in a scene but interact with light in a way that closely resembles real-life behavior. As a result, virtual objects blend naturally with their surroundings, which is a major focus in the augmented reality (AR) field. Differential Progressive Path Tracing is gaining a lot of attention because it offers a promising solution for creating more immersive and believable AR experiences, where the line between the virtual and the real becomes nearly invisible.

Lessons Learned

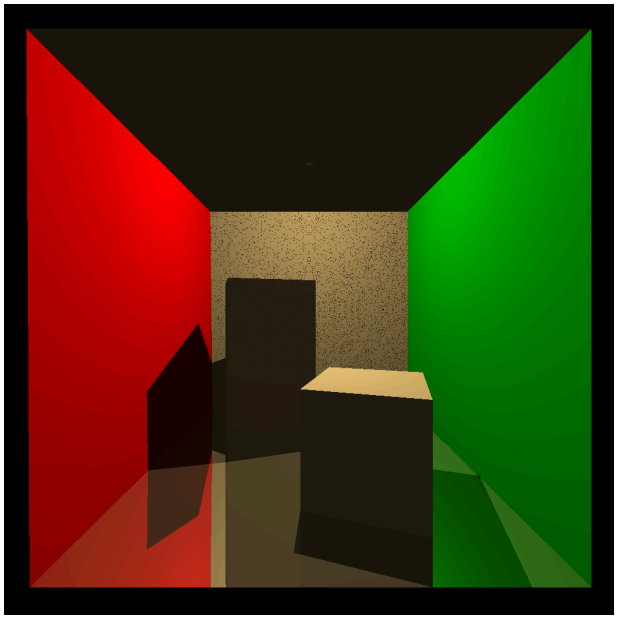
We learned several important lessons during this project. First, we found that single direction path tracing requires a very high number of rays to produce good results, making it computationally expensive and inefficient. In contrast, bidirectional tracing helps ensure that - instead of throwing out the vast majority of rays for not intersecting with the tiny light - every ray contributes to the final image and thus significantly increases the overall efficiency. We also found that noise remains a bit challenging, though it can be addressed by applying smoothing algorithms and increasing the number of rays per pixel. We learned that there are many different methods to tone mapping, each of which emphasizes different attributes of the image outputting a final product. Finally, when using only a small number of rays, the quality of the rendered output becomes largely a matter of random chance, underscoring the need for sampling to achieve consistent and reliable results.

Limitations and Future Work

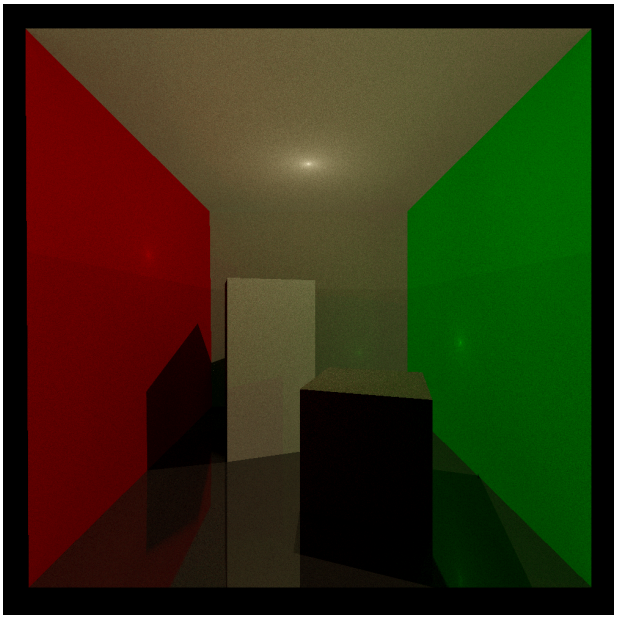
Our main limitation was time — it's too slow. We currently brute-force every path permutation per ray, and we simply didn't had enough time to optimize. Future work could include caching repeated paths, improving parallelization, and optimizing via algorithmic shortcuts. In particular, by using a non-uniform ray density and putting more rays towards locations that are more complex, you can reduce the number of rays cast while maintaining quality. For example, in adaptive sampling, the number of rays traced for each pixel can vary and the total number traced depends on the variability of the colors of the traced rays for a specific pixel. There are many other techniques too, such as importance sampling, permutation reduction, path splitting, heuristic path selection, and denoising filters.

Analysis of Results

Cornell Box Sample



Ray Traced (1000x1000)



Path Traced (1000x1000, depth 10, rpp 12, rpl 12)

The images above show the standard Cornell box being traced with the original ray tracer and our path tracer. While our result is a bit noisier, it is also significantly more physically accurate. For example, the material of the ceiling is the same as the back wall. In the ray traced scene, the ceiling is nearly black, which is not accurate. Our result shows a much more accurate ceiling that looks very similar to the back wall. While the ceiling may not be reached much directly from the lights, the box provides a lot of opportunities for light to bounce around and illuminate nearly every part of the scene. Indeed, the dark shadows in the ray traced scene are not accurate - they are also lit by scattered light, as can be seen in our result. Finally, the effect of color bleeding can be seen in our result, most prominently on the top surface of the shorter box, which looks slightly green due to the reflected green light from the right wall, and on the lower left and right sides of the back wall. While it is hard to make out, the ceiling also has a slightly red and green hue on the left and right sides, respectively.

Performance Evaluation

As shown below, even up to larger resolutions, this path tracer runs very quickly. While it does not run fast enough for real time rendering, as stated in the “State of the Art” section, there are now algorithms and hardware fast enough to do so. For smaller resolutions and smaller numbers of rays, our implementation runs nearly instantly, even for somewhat complex scenes like the Cornell box. Notably, larger RPP and RPL values rapidly scale the runtime, suggesting a focus point for future optimization. However, as seen in our sample, small RPP/RPL values can still create a high-quality output.

Resolution	RPP	RPL	Depth	Runtime (s)
100x100	1	1	1	0.131214
100x100	1	1	4	0.177773
100x100	1	1	8	0.244446
100x100	3	3	8	0.289260
100x100	5	5	8	0.476149
150x150	5	5	8	0.791814
300x300	5	5	8	1.753129
300x300	20	20	8	9.055141

Run durations of the Cornell box across multiple configurations

Processor	
Processor	AMD Ryzen 7 7700X 8-Core Processor
Number of Cores	16
Speed	5.4 GHz
Stepping	2
Family	19
Model	61
CPU ID	178BFBFF00A60F12

Memory	
RAM	32 GB

Video Card	
Video Card	AMD Radeon RX 7900 XTX
Chipset	AMD Radeon RX 7900 XTX
Manufacturer	ATI
Hardware T&L	Yes
Total Memory	40 GB
Dedicated Memory	24 GB
Driver Version	32.0.12033.1030
Vertex Shader Version	6.0
Pixel Shader Version	6.0
Plug and Play ID	VEN_1002&DEV_744C&SUBSYS_2422148C&REV_C8
Device	744C
Vendor ID	1002

Specifications of the computer used for benchmarking